LAPORAN PRAKTIK PEMROGRAMAN WEB

PEMBUATAN SERVER WEB DAN REST API SEDERHANA MENGGUNAKAN NODE.JS

Disusun Oleh:

Muhamad Fikriansyah (24260028)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK DAN ILMU KOMPUTER

UNIVERSITAS NAHDLATUL ULAMA INDONESIA

BOGOR

2026

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktik yang berjudul “Pembuatan Server Web dan REST API Sederhana Menggunakan Node.js” dengan baik. Laporan ini disusun sebagai dokumentasi praktik pemrograman web yang dilaksanakan pada tanggal 7 November 2026.

Praktik ini bertujuan memberikan pemahaman mengenai cara kerja server web dan REST API menggunakan Node.js. Dalam praktik ini, penulis membuat server menggunakan modul bawaan `http` yang menyajikan halaman profil dan lima endpoint REST API, kemudian menjalankannya di Visual Studio Code dan mengujinya melalui browser, perintah `curl`, serta skrip pengujian otomatis.

Dalam pelaksanaan praktik hingga penyusunan laporan ini, penulis memperoleh ilmu dan arahan dari dosen dan pengajar. Oleh karena itu, penulis menyampaikan terima kasih atas ilmu, arahan, dan kesempatan yang telah diberikan sehingga kegiatan praktik ini dapat terlaksana dengan baik.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi penyusunan maupun isi pembahasan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan sebagai bahan evaluasi dan perbaikan untuk penyusunan laporan selanjutnya. Penulis berharap laporan ini dapat memberikan manfaat serta menambah pemahaman mengenai pembuatan server web dan REST API dengan Node.js.

Bogor, November 2026

Penulis,

Muhamad Fikriansyah

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 1
1.3 Tujuan Praktik 1
1.4 Manfaat Praktik 2
BAB II LANDASAN TEORI 3
2.1 Node.js dan Modul Bawaan 3
2.2 Client–Server dan HTTP 3
2.3 Modul http dan Pembuatan Server 3
2.4 REST API dan Format JSON 3
2.5 Metode dan Kode Status HTTP 3
2.6 Fetch API 4
2.7 Keamanan Dasar pada Server dan Halaman Web 4
BAB III METODOLOGI DAN PERSIAPAN PRAKTIK 5
3.1 Waktu dan Tempat Pelaksanaan 5
3.2 Alat dan Bahan Praktik 5
3.3 Objek Praktik 5
3.4 Prosedur Pelaksanaan Praktik 6
BAB IV HASIL DAN PEMBAHASAN 7
4.1 Pembuatan Folder dan Struktur Proyek 7
4.2 Kode server.js dan Penjelasannya 7
4.3 Kode Frontend dan Penjelasannya 9
4.4 Menjalankan Server di Visual Studio Code 10
4.5 Hasil Tampilan di Browser 10
4.6 Pengujian REST API dengan curl 11
4.7 Pengujian Otomatis 13
4.8 Pembahasan 14
4.9 Hasil Akhir Praktik 15
BAB V PENUTUP 16
5.1 Kesimpulan 16
5.2 Saran 16
DAFTAR PUSTAKA 17

BAB I PENDAHULUAN

1.1 Latar Belakang

Halaman web statis yang hanya dibuat dengan HTML dan CSS menampilkan isi yang sama kepada setiap pengunjung. Agar halaman dapat menampilkan data yang tersimpan dan dapat berubah, misalnya data profil atau daftar target belajar, dibutuhkan sebuah server yang menerima permintaan dari browser lalu mengirimkan jawaban berupa halaman atau data. Server inilah yang menjadi inti dari sisi backend sebuah aplikasi web.

Node.js merupakan lingkungan eksekusi (runtime) JavaScript yang dibangun di atas mesin V8 sehingga JavaScript dapat dijalankan di luar browser (Node.js, n.d.-a). Node.js menyediakan modul bawaan `http` untuk membuat server web tanpa library tambahan (Node.js, n.d.-c). Data dari server umumnya dikirim melalui REST API. REST sendiri merupakan gaya arsitektur yang diperkenalkan oleh Fielding (2000), dan pada praktiknya REST API memanfaatkan metode HTTP seperti GET, POST, dan DELETE untuk mengakses sumber daya (MDN Web Docs, n.d.-b).

Berdasarkan hal tersebut, dilaksanakan praktik pembuatan server web dan REST API sederhana menggunakan Node.js pada tanggal 7 November 2026. Praktik menggunakan Visual Studio Code dengan terminal terintegrasi, serta proyek latihan bernama praktik-server-nodejs yang menyajikan halaman “Profil Junior Web Developer” dan lima endpoint API untuk data profil, teknologi, dan target belajar. Halaman tersebut mengambil datanya dari API menggunakan `fetch`, sehingga praktik ini sekaligus mempertemukan sisi frontend dan backend. Seluruh kode ditulis tanpa library tambahan agar dapat langsung dijalankan di Visual Studio Code.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam praktik ini adalah sebagai berikut:

1. Bagaimana cara kerja komunikasi antara browser (client) dan server melalui HTTP?
2. Bagaimana cara membuat server web sederhana menggunakan modul `http` pada Node.js?
3. Bagaimana cara membuat REST API dengan metode GET, POST, dan DELETE yang mengembalikan data berformat JSON?
4. Bagaimana cara menghubungkan halaman web dengan REST API menggunakan `fetch`?

5. Bagaimana cara menjalankan dan menguji server di Visual Studio Code, serta apa hasil yang diperoleh?

1.3 Tujuan Praktik

Tujuan dari pelaksanaan praktik pembuatan server web dan REST API dengan Node.js ini adalah sebagai berikut:

1. Memahami konsep client–server, request, dan response pada HTTP.
2. Mampu membuat server web sederhana menggunakan modul `http`, `fs`, dan `path` pada Node.js.
3. Mampu membuat REST API dengan metode GET, POST, dan DELETE beserta kode status HTTP yang sesuai.
4. Mampu menampilkan dan mengubah data pada halaman web menggunakan `fetch`.
5. Mampu menjalankan server di Visual Studio Code dan mengujinya melalui browser, `curl`, dan skrip pengujian otomatis.

1.4 Manfaat Praktik

Manfaat yang diperoleh dari pelaksanaan praktik ini adalah sebagai berikut:

1. **Menambah Pengetahuan:** Memberikan pemahaman mengenai cara kerja server web dan REST API.
2. **Meningkatkan Keterampilan:** Melatih penulisan kode backend dengan Node.js, penanganan kesalahan, dan pengujian API.
3. **Mempermudah Pengembangan Proyek:** Memberikan pengalaman menjalankan dan menguji proyek langsung di Visual Studio Code melalui perintah `npm start` atau tombol F5.
4. **Menambah Pengalaman Praktis:** Memberikan bekal untuk mempelajari Express.js, database, dan pengembangan aplikasi full-stack.

BAB II LANDASAN TEORI

2.1 Node.js dan Modul Bawaan

Node.js adalah lingkungan eksekusi JavaScript lintas platform yang dibangun di atas mesin V8 sehingga kode dapat dijalankan di luar browser (Node.js, n.d.-a). Node.js dilengkapi modul bawaan yang dapat dipakai langsung dengan `require()` tanpa instalasi tambahan. Tiga modul yang digunakan dalam praktik ini adalah `http` untuk membuat server (Node.js, n.d.-c), `fs` untuk membaca file (Node.js, n.d.-b), dan `path` untuk mengolah alamat file agar dapat bekerja di berbagai sistem operasi (Node.js, n.d.-d).

2.2 Client–Server dan HTTP

Komunikasi web berjalan dengan model client–server. Browser bertindak sebagai client yang mengirim permintaan (request), sedangkan server menerima permintaan itu lalu mengirim jawaban (response) (MDN Web Docs, n.d.-a). Setiap request memuat metode, alamat (URL), dan header, sedangkan setiap response memuat kode status, header, dan isi. Alamat `localhost` menunjuk ke komputer sendiri, sedangkan nomor port, misalnya 3000, menentukan program mana yang melayani permintaan.

2.3 Modul http dan Pembuatan Server

Fungsi `http.createServer()` menerima sebuah fungsi callback yang dipanggil setiap ada request. Callback tersebut menerima dua objek, yaitu `req` yang berisi metode dan alamat request, serta `res` yang dipakai untuk mengirim jawaban. Kode status dan header diatur dengan `res.writeHead()`, kemudian isi dikirim dan respons diakhiri dengan `res.end()`. Server baru mulai menerima permintaan setelah `server.listen()` dipanggil dengan nomor port tertentu (Node.js, n.d.-c).

2.4 REST API dan Format JSON

REST (Representational State Transfer) adalah gaya arsitektur yang diperkenalkan oleh Fielding (2000). Pada REST API, setiap data diperlakukan sebagai sumber daya (resource) yang memiliki alamat, misalnya `/api/target`, dan diakses menggunakan metode HTTP. Data dikirim dalam format JSON (JavaScript Object Notation), yaitu format teks berbasis pasangan kunci dan nilai yang mudah dibaca manusia dan mudah diolah oleh JavaScript melalui `JSON.stringify()` dan `JSON.parse()`.

2.5 Metode dan Kode Status HTTP

Metode HTTP menyatakan tindakan yang diminta oleh client. Metode GET digunakan untuk mengambil data, POST untuk mengirim data baru, dan DELETE untuk menghapus data (MDN Web Docs, n.d.-b). Adapun kode status menyatakan hasil pemrosesan request. Kode 2xx menandakan permintaan berhasil, 4xx menandakan kesalahan dari sisi client, dan 5xx menandakan kesalahan pada server (MDN Web Docs, n.d.-c). Kode status yang digunakan dalam praktik ini disajikan pada Tabel 2.1.

Tabel 2.1 *Kode Status HTTP yang Digunakan dalam Praktik*

Kode	Nama	Penggunaan pada praktik
200	OK	Permintaan GET dan DELETE berhasil.
201	Created	Target belajar baru berhasil dibuat melalui POST.
400	Bad Request	Isi request kosong atau bukan JSON yang valid.
403	Forbidden	Request file di luar folder `public` ditolak.
404	Not Found	Endpoint, halaman, atau id target tidak ditemukan.
405	Method Not Allowed	Metode HTTP tidak sesuai dengan endpoint.
413	Content Too Large	Isi request melebihi batas 10 KB.
500	Internal Server Error	Terjadi kesalahan tak terduga pada server.

2.6 Fetch API

Fetch API adalah antarmuka pada browser untuk mengirim request HTTP dan menerima respons secara asynchronous (MDN Web Docs, n.d.-e). Fungsi `fetch()` mengembalikan Promise sehingga dapat dipakai bersama `async` dan `await`. Properti `response.ok` bernilai true apabila kode status berada pada rentang 200 sampai 299. Pada praktik ini, `fetch` dipakai oleh file `script.js` untuk mengambil data dari API dan mengirim data baru ke server.

2.7 Keamanan Dasar pada Server dan Halaman Web

Dua risiko dasar ditangani dalam praktik ini. Pertama, path traversal, yaitu serangan yang memakai alamat seperti `../` agar server membuka file di luar folder yang seharusnya dibuka (OWASP Foundation, n.d.). Kedua, penyisipan skrip berbahaya (XSS) melalui teks masukan pengguna. Untuk mengurangi risiko ini, teks yang berasal dari pengguna ditampilkan dengan properti `textContent` karena nilainya tidak diproses sebagai HTML (MDN Web Docs, n.d.-d).

BAB III METODOLOGI DAN PERSIAPAN PRAKTIK

3.1 Waktu dan Tempat Pelaksanaan

Praktik pembuatan server web dan REST API dengan Node.js dilaksanakan pada:

Hari/Tanggal : Sabtu, 7 November 2026

Tempat : Komputer/laptop pribadi penulis

Kegiatan : Praktik Pemrograman Web

Materi : Server Web dan REST API Sederhana dengan Node.js

Praktik dilakukan menggunakan Visual Studio Code dengan terminal terintegrasi untuk menulis dan menjalankan kode, serta browser untuk memeriksa tampilan halaman.

3.2 Alat dan Bahan Praktik

Dalam pelaksanaan praktik, diperlukan beberapa alat dan bahan untuk mendukung penulisan dan pengujian program. Adapun alat dan bahan yang digunakan adalah sebagai berikut:

1. **Perangkat Keras (Hardware)**
   - Komputer/laptop pribadi sebagai perangkat utama untuk melaksanakan praktik.
2. **Perangkat Lunak (Software)**
   - **Visual Studio Code:** Editor kode untuk menulis program sekaligus tempat menjalankan terminal terintegrasi.
   - **Node.js (versi 22.23.4):** Runtime untuk menjalankan server JavaScript di luar browser.
   - **npm:** Pengelola paket bawaan Node.js yang dipakai untuk menjalankan skrip `npm start`, `npm run dev`, dan `npm test`.
   - **Peramban web (browser):** Untuk membuka halaman dan memeriksa tampilannya.
   - **curl:** Perangkat baris perintah untuk mengirim request HTTP ke API dari terminal.
3. **Bahan Praktik**
   - Folder proyek: praktik-server-nodejs.
   - File program: `server.js` dan `test-api.js`.
   - File frontend di folder `public`: `index.html`, `style.css`, dan `script.js`.
   - File pendukung: `package.json`, `.vscode/launch.json`, `requests.http`, dan `README.md`.

3.3 Objek Praktik

Objek yang digunakan dalam praktik ini adalah sebuah aplikasi web sederhana bernama “Profil Junior Web Developer” yang terdiri dari sisi server dan sisi client. Sisi server berupa file `server.js` sepanjang 205 baris yang menyajikan file halaman dan lima endpoint REST API. Sisi client berupa halaman `index.html` (49 baris), `style.css` (69 baris), dan `script.js` (85 baris) yang mengambil data dari API dan menampilkannya di browser.

Data disimpan dalam variabel di memori sehingga tidak memerlukan database. Selain itu, tersedia file `test-api.js` (125 baris) yang menjalankan 14 pengujian otomatis terhadap server. Seluruh kode hanya menggunakan modul bawaan Node.js sehingga tidak diperlukan perintah `npm install` dan proyek dapat langsung dijalankan di Visual Studio Code.

3.4 Prosedur Pelaksanaan Praktik

Prosedur pelaksanaan praktik dilakukan melalui beberapa tahapan secara berurutan, mulai dari pembuatan folder proyek hingga pengujian otomatis. Adapun prosedurnya adalah sebagai berikut:

1. **Pembuatan Folder Proyek:** Membuat folder praktik-server-nodejs lalu membukanya di Visual Studio Code melalui menu File > Open Folder.
2. **Pembuatan package.json:** Membuat file konfigurasi proyek yang berisi skrip `start`, `dev`, dan `test`.
3. **Penulisan server.js:** Menulis kode server yang berisi data, fungsi bantu, penanganan REST API, dan penyaji file statis.
4. **Pembuatan Frontend:** Membuat file `index.html`, `style.css`, dan `script.js` di dalam folder `public`.
5. **Konfigurasi Visual Studio Code:** Membuat `.vscode/launch.json` agar server dapat dijalankan dengan tombol F5.
6. **Menjalankan Server:** Menjalankan perintah `npm start` pada terminal terintegrasi atau menekan F5.
7. **Pemeriksaan di Browser:** Membuka alamat http://localhost:3000 dan mencoba menambah serta menghapus target belajar.
8. **Pengujian API dengan curl:** Mengirim request ke setiap endpoint dan memeriksa log pada terminal server.
9. **Pengujian Otomatis:** Menjalankan `npm test` untuk memeriksa seluruh endpoint secara otomatis.

BAB IV HASIL DAN PEMBAHASAN

4.1 Pembuatan Folder dan Struktur Proyek

Tahap pertama adalah membuat folder proyek dan membukanya di Visual Studio Code. Seluruh file dipisahkan berdasarkan fungsinya, yaitu kode server berada di folder utama, sedangkan file yang diakses browser dikumpulkan di folder `public`. Pemisahan ini penting agar server hanya membuka file di folder `public` dan tidak membuka file kode server. Struktur folder proyek ditunjukkan sebagai berikut:

```
praktik-server-nodejs/
├── .vscode/
│   └── launch.json
├── public/
│   ├── index.html
│   ├── script.js
│   └── style.css
├── package.json
├── README.md
├── requests.http
├── server.js
└── test-api.js
```

File `package.json` berisi tiga skrip, yaitu `start` untuk menjalankan `node server.js`, `dev` untuk menjalankan `node --watch server.js` yang memulai ulang server otomatis saat kode diubah, serta `test` untuk menjalankan `node test-api.js`.

4.2 Kode server.js dan Penjelasannya

File `server.js` terdiri dari 205 baris dan dibagi ke dalam beberapa bagian yang diberi komentar pemisah. Penjelasan setiap bagian disajikan pada Tabel 4.1.

Tabel 4.1 *Penjelasan Bagian-Bagian server.js*

No	Bagian	Baris	Penjelasan
1	Impor modul dan konfigurasi	10–26	Memuat modul `http`, `fs`, `path`; menetapkan port, folder `public`, dan tipe MIME.
2	Data di memori	28–44	Menyimpan data `profil`, `teknologi`, dan `daftarTarget` pada variabel.
3	Fungsi bantu	46–83	`kirimJson()`, `kirimTeks()`, dan `bacaBody()` untuk mengirim respons dan membaca isi request.
4	Penanganan REST API	85–139	Fungsi `tanganiApi()` yang menangani lima endpoint beserta validasi dan kode status.
5	Penyaji file statis	141–161	Fungsi `kirimFile()` yang membaca file dari folder `public` dan menolak akses di luar folder.
6	Pembuatan server	163–185	`http.createServer()` yang mencatat log dan memisahkan rute API dari file statis.

No	Bagian	Baris	Penjelasan
7	Menjalankan dan mengekspor server	187–205	`server.listen()` hanya saat file dijalankan langsung; `module.exports` agar dapat diuji.

Server dibuat dengan `http.createServer()`. Pada setiap request, alamat dibaca menggunakan `new URL()`. Jika alamat diawali `/api/`, request diserahkan ke fungsi `tanganiApi()`; selain itu server mencoba menyajikan file dari folder `public`. Potongan kode pemisah rute tersebut adalah sebagai berikut:

```
try {
  const { pathname } = new URL(req.url, "http://localhost");
  if (pathname.startsWith("/api/")) {
    await tanganiApi(req, res, pathname);
  } else if (req.method === "GET" || req.method === "HEAD") {
    kirimFile(res, pathname);
  } else {
    kirimTeks(res, 405, "405 - Metode tidak diizinkan");
  }
```

Endpoint untuk menambah target belajar menggunakan metode POST. Server membaca isi request, mengubahnya dari JSON menjadi objek, memeriksa bahwa field `judul` tidak kosong, lalu menyimpannya dengan id baru dan membalas dengan kode 201. Jika isi request tidak valid, server membalas dengan kode 400. Potongan kodenya adalah sebagai berikut:

```
if (metode === "POST") {
  let body;
  try {
    body = JSON.parse(await bacaBody(req));
  } catch (err) {
    if (err.status) return kirimJson(res, err.status, { pesan: err.message });
    return kirimJson(res, 400, { pesan: "Body harus berupa JSON yang valid" });
  }
  const judul = body && typeof body.judul === "string" ? body.judul.trim() : "";
  if (!judul) {
    return kirimJson(res, 400, { pesan: "Field 'judul' wajib diisi" });
  }
  const baru = { id: idBerikutnya++, judul };
  daftarTarget.push(baru);
  return kirimJson(res, 201, baru);
}
```

Server juga membatasi file yang boleh dibuka. Alamat file digabungkan dengan folder `public`, dinormalisasi dengan `path.normalize()`, lalu diperiksa apakah hasilnya masih berada di dalam folder `public`. Jika tidak, server membalas dengan kode 403 sehingga alamat seperti `../server.js` tidak dapat dipakai untuk membuka kode server.

```
const lengkap = path.normalize(path.join(PUBLIC_DIR, relatif));
if (!lengkap.startsWith(PUBLIC_DIR + path.sep)) {
  return kirimTeks(res, 403, "403 - Akses ditolak");
}
```

4.3 Kode Frontend dan Penjelasannya

Halaman `index.html` disusun dengan tag semantik `main`, `section`, dan `footer`. Setiap bagian halaman berada di dalam `section` tersendiri, yaitu judul utama, Tentang Saya, Teknologi yang Dipelajari, dan Target Belajar. Bagian isi seperti nama dan daftar teknologi sengaja dikosongkan di HTML karena akan diisi oleh JavaScript setelah data diambil dari API. Potongan bagian Target Belajar adalah sebagai berikut:

```
<section>
  <h2>Target Belajar</h2>
  <ol id="daftar-target"></ol>
  <form id="form-target" autocomplete="off">
    <input type="text" id="input-target" maxlength="100" required
           placeholder="Tambah target belajar baru...">
    <button type="submit">Tambah</button>
  </form>
  <p id="status" role="status" aria-live="polite"></p>
</section>
```

File `script.js` berisi fungsi `ambilJson()` yang membungkus `fetch`. Fungsi ini mengubah respons menjadi objek dan melemparkan kesalahan apabila `res.ok` bernilai false, sehingga pesan kesalahan dari server dapat ditampilkan kepada pengguna.

```
async function ambilJson(url, opsi) {
  const res = await fetch(url, opsi);
  const data = await res.json();
  if (!res.ok) throw new Error(data.pesan || "Terjadi kesalahan");
  return data;
}
```

Saat formulir dikirim, `script.js` mengirim request POST berisi JSON ke `/api/target`, lalu memuat ulang daftar target. Nama target ditampilkan dengan `textContent`, bukan `innerHTML`, sehingga teks masukan tidak dijalankan sebagai kode HTML.

```
document.getElementById("form-target").addEventListener("submit", async (e) => {
  e.preventDefault();
  const input = document.getElementById("input-target");
  try {
    const baru = await ambilJson("/api/target", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ judul: input.value }),
    });
    tampilkanStatus(`Target "${baru.judul}" berhasil ditambahkan.`);
    input.value = "";
    await muatTarget();
  } catch (err) {
    tampilkanStatus(err.message, true);
  }
```

4.4 Menjalankan Server di Visual Studio Code

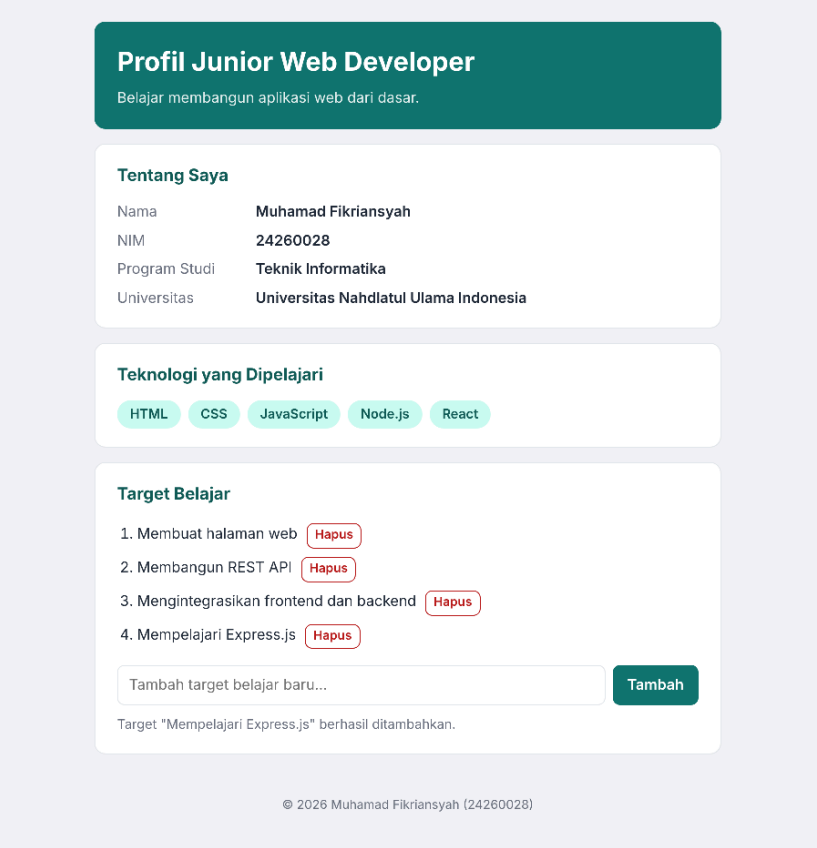
Setelah seluruh file dibuat, server dijalankan dari Visual Studio Code dengan dua cara. Cara pertama adalah membuka terminal terintegrasi melalui menu Terminal > New Terminal lalu mengetik `npm start`. Cara kedua adalah menekan tombol F5 dan memilih konfigurasi “Jalankan Server (F5)” yang tersimpan pada `.vscode/launch.json`. Kedua cara menjalankan perintah yang sama, yaitu `node server.js`, dan menghasilkan keluaran berikut di terminal:

```
> node server.js
Server berjalan di http://localhost:3000
Tekan Ctrl + C untuk menghentikan server.
```

Pesan tersebut menandakan bahwa server telah berjalan dan siap menerima request pada port 3000. Server dihentikan dengan menekan Ctrl + C pada terminal.

4.5 Hasil Tampilan di Browser

Setelah server berjalan, alamat http://localhost:3000 dibuka di browser. Halaman tampil dari atas ke bawah, yaitu judul “Profil Junior Web Developer”, bagian Tentang Saya yang berisi nama, NIM, program studi, dan universitas, bagian Teknologi yang Dipelajari berisi lima teknologi, bagian Target Belajar berisi daftar target beserta formulir, dan footer hak cipta. Seluruh data pada halaman diambil dari API, bukan ditulis langsung di HTML. Pada Gambar 4.1, satu target baru bernama “Mempelajari Express.js” telah ditambahkan melalui formulir sehingga daftar menjadi empat target dan pesan keberhasilan muncul di bawah formulir.



Gambar 4.1 *Tampilan Halaman Profil Junior Web Developer di Browser*

4.6 Pengujian REST API dengan curl

Setiap endpoint diuji dengan mengirim request menggunakan perintah `curl`. Pengujian pertama adalah mengambil data profil dengan opsi `-i` agar header respons ikut ditampilkan:

```
> curl -i http://localhost:3000/api/profil
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 198
Date: Sat, 07 Nov 2026 02:50:25 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{
  "nama": "Muhamad Fikriansyah",
  "nim": "24260028",
  "prodi": "Teknik Informatika",
  "fakultas": "Fakultas Teknik dan Ilmu Komputer",
  "universitas": "Universitas Nahdlatul Ulama Indonesia"
}
```

Pengujian berikutnya adalah menambah target belajar dengan metode POST, kemudian mengirim judul kosong untuk memeriksa validasi. Request pertama berhasil dan mengembalikan data target dengan id 4, sedangkan request kedua ditolak dengan kode 400:

```
> curl -X POST http://localhost:3000/api/target \
    -H "Content-Type: application/json" \
    -d '{"judul":"Mempelajari Express.js"}'
{
  "id": 4,
  "judul": "Mempelajari Express.js"
}

> curl -X POST http://localhost:3000/api/target \
    -H "Content-Type: application/json" \
    -d '{"judul":""}'
{
  "pesan": "Field 'judul' wajib diisi"
}
HTTP 400
```

Hasil seluruh pengujian dengan `curl` dirangkum pada Tabel 4.2. Setiap kode status yang diterima sesuai dengan yang diharapkan.

Tabel 4.2 *Hasil Pengujian Endpoint dengan curl*

No	Permintaan	Status	Keterangan
1	GET /api/profil	200 OK	Mengembalikan data profil dalam format JSON.
2	POST /api/target (judul terisi)	201 Created	Target baru dibuat dengan id 4.
3	POST /api/target (judul kosong)	400 Bad Request	Ditolak: field `judul` wajib diisi.
4	GET /api/target	200 OK	Mengembalikan empat target, termasuk id 4.
5	DELETE /api/target/4	200 OK	Target dengan id 4 berhasil dihapus.
6	GET /halaman-tidak-ada	404 Not Found	Halaman tidak ditemukan.

Selama pengujian, server mencatat setiap request pada terminal berupa jam, metode, alamat, dan kode status. Log tersebut sesuai dengan urutan request pada Tabel 4.2:

```
[02.50.25] GET /api/profil -> 200
[02.50.25] POST /api/target -> 201
[02.50.25] POST /api/target -> 400
[02.50.25] GET /api/target -> 200
[02.50.25] DELETE /api/target/4 -> 200
[02.50.25] GET /halaman-tidak-ada -> 404
```

4.7 Pengujian Otomatis

Pengujian manual kurang praktis jika diulang setiap kali kode diubah. Oleh karena itu dibuat file `test-api.js` yang menjalankan server pada port acak, mengirim request ke seluruh endpoint menggunakan `fetch`, lalu memeriksa kode status dan isi respons. File

ini dijalankan dengan perintah `npm test` (atau `node test-api.js`) dan menghasilkan keluaran berikut:

```
> node test-api.js
Menguji server pada http://127.0.0.1:37313

[OK]    GET    /api/profil            -> 200, data Muhamad Fikriansyah
[OK]    GET    /api/teknologi         -> 200, 5 teknologi
[OK]    GET    /api/target            -> 200, 3 target awal
[OK]    POST   /api/target            -> 201, target baru dibuat
[OK]    POST   /api/target (kosong)   -> 400, ditolak
[OK]    POST   /api/target (rusak)   -> 400, ditolak
[OK]    GET    /api/target            -> 200, sekarang 4 target
[OK]    DELETE /api/target/4          -> 200, target dihapus
[OK]    DELETE /api/target/999        -> 404, tidak ditemukan
[OK]    GET    /api/tidak-ada         -> 404
[OK]    PUT    /api/profil            -> 405, metode ditolak
[OK]    GET    /                      -> 200, halaman HTML
[OK]    GET    /style.css             -> 200, berkas CSS
[OK]    GET    /..%2fserver.js        -> 403, akses di luar public ditolak

Ringkasan: 14 lulus, 0 gagal dari 14 pengujian.
```

Keluaran menunjukkan bahwa keempat belas pengujian lulus. Pengujian mencakup kasus berhasil (200 dan 201), kasus input salah (400), data tidak ditemukan (404), metode tidak diizinkan (405), serta akses di luar folder `public` yang ditolak (403).

4.8 Pembahasan

Hasil praktik menunjukkan bahwa server web dan REST API dapat dibuat hanya dengan modul bawaan Node.js. Pemisahan rute berdasarkan awalan `/api/` membuat kode lebih tertata, karena permintaan data ditangani oleh `tanganiApi()` dan permintaan halaman ditangani oleh `kirimFile()`.

Penggunaan kode status yang tepat membuat respons server mudah dipahami oleh client. Kode 201 dipakai ketika data baru berhasil dibuat, 400 ketika masukan tidak valid, 404 ketika data tidak ditemukan, dan 405 ketika metode tidak sesuai. Pengujian `GET /..%2fserver.js` yang menghasilkan kode 403 membuktikan bahwa pembatasan akses file bekerja sehingga kode server tidak dapat dibuka dari browser.

Dari sisi keamanan, penggunaan `textContent` pada `script.js` mencegah teks masukan pengguna dijalankan sebagai HTML. Adapun kekurangan program ini adalah data hanya disimpan di memori. Akibatnya, target yang ditambahkan akan hilang ketika server dimatikan, dan daftar kembali ke tiga target awal saat server dijalankan ulang. Kondisi inilah yang menjadi dasar angka pada pengujian otomatis, yaitu tiga target awal sebelum penambahan.

Fungsi `server.listen()` pada `server.js` hanya dipanggil bila file dijalankan langsung, yang dicek dengan `require.main === module`. Dengan demikian

`test-api.js` dapat memuat `server.js` dan menentukan port sendiri tanpa bentrok dengan server yang sedang berjalan di port 3000.

4.9 Hasil Akhir Praktik

Berdasarkan seluruh tahapan praktik, server web dan REST API berhasil dibuat, dijalankan di Visual Studio Code, dan lulus seluruh pengujian. Ringkasan perintah dan cara menjalankan yang digunakan dalam praktik ini disajikan pada Tabel 4.3.

Tabel 4.3 *Ringkasan Perintah yang Digunakan*

No	Perintah	Fungsi
1	npm start	Menjalankan server pada http://localhost:3000.
2	F5 (Visual Studio Code)	Menjalankan server melalui konfigurasi `.vscode/launch.json`.
3	npm run dev	Menjalankan server dengan mulai ulang otomatis saat kode berubah.
4	npm test	Menjalankan 14 pengujian otomatis pada REST API.
5	curl -i alamat	Mengirim request HTTP ke API dan menampilkan header respons.
6	Ctrl + C	Menghentikan server yang sedang berjalan di terminal.

BAB V PENUTUP

5.1 Kesimpulan

Berdasarkan praktik yang telah dilaksanakan, dapat disimpulkan bahwa server web dan REST API sederhana berhasil dibuat menggunakan modul bawaan `http` pada Node.js dan dapat dijalankan langsung di Visual Studio Code dengan perintah `npm start` atau tombol F5. Server berhasil menyajikan halaman profil serta lima endpoint dengan metode GET, POST, dan DELETE yang menggunakan kode status HTTP yang sesuai.

Hasil praktik juga menunjukkan bahwa halaman web berhasil mengambil dan mengubah data melalui `fetch`, dan seluruh empat belas pengujian otomatis dinyatakan lulus. Validasi masukan, pembatasan akses file, dan penggunaan `textContent` membuat aplikasi lebih aman. Keterbatasan yang ditemui adalah data masih disimpan di memori sehingga kembali ke kondisi awal setiap server dijalankan ulang.

5.2 Saran

Praktik selanjutnya dapat dikembangkan dengan menyimpan data ke database, misalnya SQLite atau MongoDB, agar data tidak hilang saat server dimatikan. Selain itu, dapat ditambahkan endpoint dengan metode PUT untuk mengubah data yang sudah ada.

Penulis juga menyarankan penggunaan framework Express.js untuk menyederhanakan penanganan rute ketika jumlah endpoint bertambah banyak, serta penyimpanan proyek ke GitHub menggunakan perintah git add, git commit, dan git push agar riwayat perubahan terdokumentasi dengan baik.

DAFTAR PUSTAKA

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

MDN Web Docs. (n.d.-a). *An overview of HTTP*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview

MDN Web Docs. (n.d.-b). *HTTP request methods*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods

MDN Web Docs. (n.d.-c). *HTTP response status codes*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

MDN Web Docs. (n.d.-d). *Node: textContent property*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Node/textContent

MDN Web Docs. (n.d.-e). *Using the Fetch API*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

Node.js. (n.d.-a). *About Node.js*. OpenJS Foundation. https://nodejs.org/en/about

Node.js. (n.d.-b). *File system*. Node.js documentation. https://nodejs.org/api/fs.html

Node.js. (n.d.-c). *HTTP*. Node.js documentation. https://nodejs.org/api/http.html

Node.js. (n.d.-d). *Path*. Node.js documentation. https://nodejs.org/api/path.html

OWASP Foundation. (n.d.). *Path traversal*. https://owasp.org/www-community/attacks/Path_Traversal